

Structure of Programming Language

Using Python

Python Files and Exception Handling

How Python Handle Files?

If you are working in a large software application where they process a large number of data, then we cannot expect those data to be stored in a variable as the variables are volatile in nature.

Hence when are you about to handle such situations, the role of files will come into the picture.

As files are non-volatile in nature, the data will be stored permanently in a secondary device like Hard Disk and using python we will handle these files in our applications

Are you thinking about how python will handle files?

Let's take an **Example** of how normal people will handle the files. If we want to read the data from a file or write the data into a file, then, first of all, we will open the file or will create a new file if the file does not exist and then perform the normal read/write operations, save the file and close it.

Types Of File in Python

There are two types of files in Python and each of them are explained below in detail with examples for your easy understanding.

They are:

- Binary file
- Text file

Binary files in Python

Most of the files that we see in our computer system are called binary files.

Example:

Document files: .pdf, .doc, .xls etc.

Image files: .png, .jpg, .gif, .bmp etc.

Video files: .mp4, .3gp, .mkv, .avi etc.

Audio files: .mp3, .wav, .mka, .aac etc.

Database files: .mdb, .accde, .frm, .sqlite etc.

Archive files: .zip, .rar, .iso, .7z etc.

Executable files: .exe, .dll, .class etc.

All binary files follow a specific format. We can open some binary files in the normal text editor but we can't read the content present inside the file. That's because all the binary files will be encoded in the binary format, which can be understood only by a computer or machine.

For handling such binary files we need a specific type of software to open it.

For Example

You need Microsoft word software to open .doc binary files. Likewise, you need a pdf reader software to open .pdf binary files and you need a photo editor software to read the image files and so on.

Text files in Python

Text files don't have any specific encoding and it can be opened in normal text editor itself.

Example:

Web standards: html, XML, CSS, JSON etc.

Source code: c, app, js, py, java etc.

Documents: txt, tex, RTF etc.

Tabular data: csv, tsv etc.

Configuration: ini, cfg, reg etc.

Python File Handling Operations

Most importantly there are 4 types of operations that can be handled by Python on files:

- Open
- Read
- Write
- Close

Other operations include:

- Rename
- Delete

Python Files I/O

Printing to the Screen:

The simplest way to produce output is using the *print* statement where you can pass zero or more expressions, separated by commas. This function converts the expressions you pass it to a string and writes the result to standard output as follows:

```
print "Python is really a great language,", "isn't it?";
```

This would produce following result on your standard screen:

```
Python is really a great language, isn't it?
```

Reading Keyboard Input:

Reading Keyboard Input:

Python provides two built-in functions to read a line of text from standard input, which by default comes from the keyboard. These functions are:

`raw_input`

`input`

The *raw_input* Function:

The *raw_input([prompt])* function reads one line from standard input and returns it as a string (removing the trailing newline):

```
str = raw_input("Enter your input: ");
```

```
print "Received input is : ", str
```

This would prompt you to enter any string and it would display same string on the screen.

When I typed "Hello Python!", it output is like this:

```
Enter your input: Hello Python
```

```
Received input is : Hello Python
```

Reading Keyboard Input:

The *input* Function:

The *input([prompt])* function is equivalent to `raw_input`, except that it assumes the input is a valid Python expression and returns the evaluated result to you:

```
str = input("Enter your input: ");  
print "Received input is : ", str
```

This would produce following result against the entered input:

```
Enter your input: [x*5 for x in range(2,10,2)]  
Recieved input is : [10, 20, 30, 40]
```

Opening and Closing Files

Until now, you have been reading and writing to the standard input and output. Now we will see how to play with actual data files.

Python provides basic functions and methods necessary to manipulate files by default. You can do your most of the file manipulation using a file object.

The *open* Function:

Before you can read or write a file, you have to open it using Python's built-in *open()* function. This function creates a file object which would be utilized to call other support methods associated with it.

Syntax:

```
file object = open(file_name [, access_mode][, buffering])
```

Parameters detail:

- **file_name:** The file_name argument is a string value that contains the name of the file that you want to access.
- **access_mode:** The access_mode determines the mode in which the file has to be opened ie. read, write append etc. A complete list of possible values is given below in the table. This is optional parameter and the default file access mode is read (r)
- **buffering:** If the buffering value is set to 0, no buffering will take place. If the buffering value is 1, line buffering will be performed while accessing a file. If you specify the buffering value as an integer greater than 1, then buffering action will be performed with the indicated buffer size. If negative, the buffer size is the system default(default behavior).

A list of the different modes of opening a file:

Modes	Description
r	Opens a file for reading only. The file pointer is placed at the beginning of the file. This is the default mode.
rb	Opens a file for reading only in binary format. The file pointer is placed at the beginning of the file. This is the default mode.
r+	Opens a file for both reading and writing. The file pointer will be at the beginning of the file.
rb+	Opens a file for both reading and writing in binary format. The file pointer will be at the beginning of the file.
w	Opens a file for writing only. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
wb	Opens a file for writing only in binary format. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
w+	Opens a file for both writing and reading. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.

A list of the different modes of opening a file:

wb+	Opens a file for both writing and reading in binary format. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
a	Opens a file for appending. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
ab	Opens a file for appending in binary format. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
a+	Opens a file for both appending and reading. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.
ab+	Opens a file for both appending and reading in binary format. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.

The *file* object attributes:

Once a file is opened and you have one *file* object, you can get various information related to that file.

Here is a list of all attributes related to file object:

Attribute	Description
file.closed	Returns true if file is closed, false otherwise.
file.mode	Returns access mode with which file was opened.
file.name	Returns name of the file.
file.softspace	Returns false if space explicitly required with print, true otherwise.



Example

Example:

```
fo = open("test.txt", "wb")  
print "Name of the file: ", fo.name  
print "Closed or not : ", fo.closed  
print "Opening mode : ", fo.mode  
print "Softspace flag : ", fo.softspace
```

This would produce following result:

```
Name of the file: test.txt  
Closed or not : False  
Opening mode : wb  
Softspace flag : 0
```

The `close()` Method:

The `close()` method of a *file* object flushes any unwritten information and closes the file object, after which no more writing can be done.

Python automatically closes a file when the reference object of a file is reassigned to another file. It is a good practice to use the `close()` method to close a file.

Syntax:

```
fileObject.close();
```

Example:

```
fo = open("test.txt", "wb")  
print "Name of the file: ", fo.name  
fo.close()
```

This would produce following result:

```
Name of the file: test.txt
```


Reading and Writing Files:

The *file* object provides a set of access methods to make our lives easier. We would see how to use *read()* and *write()* methods to read and write files.

The *write()* Method:

- The *write()* method writes any string to an open file. It is important to note that Python strings can have binary data and not just text.
- The *write()* method does not add a newline character ('\n') to the end of the string:

Syntax:

```
fileObject.write(string);
```

Example

Python Code

```
fo = open("test.txt", "wb")  
fo.write( "Python is a great language.\r\nYeah its great!!\r\n");  
fo.close()
```

The above method would create *foo.txt* file and would write given content in that file and finally it would close that file. If you would open this file, it would have following content

```
Python is a great language.  
Yeah its great!!
```

The *read()* Method:

The *read()* Method:

The *read()* method read a string from an open file. It is important to note that Python strings can have binary data and not just text.

Syntax:

```
fileObject.read([count]);
```

Here passed parameter is the number of bytes to be read from the opened file. This method starts reading from the beginning of the file and if *count* is missing then it tries to read as much as possible, may be until the end of file.

Example:

```
fo = open("test.txt", "r+")
str = fo.read(10);
print "Read String is : ", str
fo.close()
```

This would produce following result:

Read String is : Python is

File Positions:

The *tell()* method tells you the current position within the file in other words, the next read or write will occur at that many bytes from the beginning of the file:

The *seek(offset[, from])* method changes the current file position. The *offset* argument indicates the number of bytes to be moved. The *from* argument specifies the reference position from where the bytes are to be moved.

If *from* is set to 0, it means use the beginning of the file as the reference position and 1 means use the current position as the reference position and if it is set to 2 then the end of the file would be taken as the reference position.

Example:

Python code

```
fo = open("test.txt", "r+")
str = fo.read(10);
print "Read String is : ", str
position = fo.tell();
print "Current file position : ", position
position = fo.seek(0, 0);
str = fo.read(10);
print "Again read String is : ", str
fo.close()
```

This would produce following result:

```
Read String is : Python is
Current file position : 10
Again read String is : Python is
```

Renaming and Deleting Files:

Python os module provides methods that help you perform file-processing operations, such as renaming and deleting files.

To use this module you need to import it first and then you can call any related functions.

The `rename()` Method:

The *rename()* method takes two arguments, the current filename and the new filename.

Syntax:

```
os.rename(current_file_name, new_file_name)
```

Example:

```
import os
os.rename( "test1.txt", "test2.txt" )
```

The *delete()* Method:

You can use the *delete()* method to delete files by supplying the name of the file to be deleted as the argument.

Syntax:

```
os.remove(file_name)
```

Example:

```
import os  
os.remove("test2.txt")
```

Directories in Python:

All files are contained within various directories, and Python has no problem handling these too. The `os` module has several methods that help you create, remove, and change directories.

The *mkdir()* Method:

You can use the *mkdir()* method of the `os` module to create directories in the current directory. You need to supply an argument to this method, which contains the name of the directory to be created.

Syntax:

```
os.mkdir("newdir")
```

Example:

```
import os # Create a directory "test"  
os.mkdir("test")
```

The *chdir()* Method:

You can use the *chdir()* method to change the current directory. The *chdir()* method takes an argument, which is the name of the directory that you want to make the current directory.

Syntax:

```
os.chdir("newdir")
```

Example:

```
import os  
os.chdir("/home/newdir")
```


The `getcwd()` Method:

The `getcwd()` method displays the current working directory.

Syntax:

```
os.getcwd()
```

Example:

```
import os  
os.getcwd()
```

The *rmdir()* Method:

The *rmdir()* method deletes the directory, which is passed as an argument in the method.

Before removing a directory, all the contents in it should be removed.

Syntax:

```
os.rmdir( 'dirname' )
```

Example:

```
import os  
os.rmdir( "/tmp/test" )
```

File & Directory Related Methods:

File & Directory Related Methods:

There are three important sources which provide a wide range of utility methods to handle and manipulate files & directories on Windows and Unix operating systems. They are as follows:

- [File Object Methods](#): The *file* object provides functions to manipulate files.
- [OS Object Methods](#): This provides methods to process files as well as directories.

Python Files and Exception Handling

Python Exceptions Handling

Python provides two very important features to handle any unexpected error in your Python programs and to add debugging capabilities in them:

- Exception Handling
- Assertions

What is Exception?

- An exception is an event, which occurs during the execution of a program, that disrupts the normal flow of the program's instructions.
- In general, when a Python script encounters a situation that it can't cope with, it raises an exception. An exception is a Python object that represents an error.
- When a Python script raises an exception, it must either handle the exception immediately otherwise it would terminate and come out.

Handling an exception:

Handling an exception:

If you have some *suspicious* code that may raise an exception, you can defend your program by placing the suspicious code in a try: block. After the try: block, include an except: statement, followed by a block of code which handles the problem as elegantly as possible.

Syntax:

```
try:
```

```
    You do your operations here;
```

```
    .....
```

```
except Exception I:
```

```
    If there is ExceptionI, then execute this block.
```

```
except Exception II:
```

```
    If there is ExceptionII, then execute this block.
```

```
    .....
```

```
else:
```

```
    If there is no exception then execute this block.
```

Here are few important points above the above mentioned syntax:

- A single try statement can have multiple except statements. This is useful when the try block contains statements that may throw different types of exceptions.
- You can also provide a generic except clause, which handles any exception.
- After the except clause(s), you can include an else-clause. The code in the else-block executes if the code in the try: block does not raise an exception.
- The else-block is a good place for code that does not need the try: block's protection.

Example:

```
try:  
    fh = open("testfile", "w")  
    fh.write("This is my test file for exception handling!!")  
except IOError: print "Error: can\'t find file or read data"  
else: print "Written content in the file successfully"  
fh.close()
```

This will produce following result:

```
Written content in the file successfully
```

The *except* clause with no exceptions:

You can also use the `except` statement with no exceptions defined as follows:

```
try:
```

```
    You do your operations here;
```

```
    .....
```

```
except:
```

```
    If there is any exception, then execute this block.
```

```
    .....
```

```
else:
```

```
    If there is no exception then execute this block.
```

This kind of a try-except statement catches all the exceptions that occur. Using this kind of try-except statement is not considered a good programming practice, though, because it catches all exceptions but does not make the programmer identify the root cause of the problem that may occur.

The *except* clause with multiple exceptions:

You can also use the same *except* statement to handle multiple exceptions as follows:

```
try:
```

```
    You do your operations here;
```

```
    .....
```

```
except(Exception1[, Exception2[,...ExceptionN]]):
```

```
    If there is any exception from the given exception list, then execute  
    this block
```

```
    .....
```

```
else:
```

```
    If there is no exception then execute this block.
```

The try-finally clause:

You can use a **finally:** block along with a **try:** block. The **finally** block is a place to put any code that must execute, whether the **try**-block raised an exception or not. The syntax of the **try-finally** statement is this:

try:

You do your operations here;

.....

Due to any exception, this may be skipped.

finally:

This would always be executed.

.....

Note that you can provide except clause(s), or a finally clause, but not both. You can not use *else* clause as well along with a finally clause.

Example:

```
try:
```

```
    fh = open("testfile", "w")
```

```
    fh.write("This is my test file for exception handling!!")
```

```
finally:
```

```
    print "Error: can\'t find file or read data"
```

If you do not have permission to open the file in writing mode then this will produce following result:

```
Error: can't find file or read data
```

Python Standard Exceptions

EXCEPTION NAME	DESCRIPTION
Exception	Base class for all exceptions
StopIteration	Raised when the next() method of an iterator does not point to any object.
SystemExit	Raised by the sys.exit() function.
StandardError	Base class for all built-in exceptions except StopIteration and SystemExit.
ArithmeticError	Base class for all errors that occur for numeric calculation.
OverflowError	Raised when a calculation exceeds maximum limit for a numeric type.
FloatingPointError	Raised when a floating point calculation fails.
ZeroDivisonError	Raised when division or modulo by zero takes place for all numeric types.
AssertionError	Raised in case of failure of the Assert statement.
AttributeError	Raised in case of failure of attribute reference or assignment.

EOFError	Raised when there is no input from either the raw_input() or input() function and the end of file is reached.
ImportError	Raised when an import statement fails.
KeyboardInterrupt	Raised when the user interrupts program execution, usually by pressing Ctrl+c.
LookupError	Base class for all lookup errors.
IndexError	Raised when an index is not found in a sequence.
KeyError	Raised when the specified key is not found in the dictionary.
NameError	Raised when an identifier is not found in the local or global namespace.
UnboundLocalError	Raised when trying to access a local variable in a function or method but no value has been assigned to it.
EnvironmentError	Base class for all exceptions that occur outside the Python environment.
IOError	Raised when an input/ output operation fails, such as the print statement or the open() function when trying to open a file that does not exist.

Python Standard Exceptions

OSError	Raised for operating system-related errors.
SyntaxError	Raised when there is an error in Python syntax.
IndentationError	Raised when indentation is not specified properly.
SystemError	Raised when the interpreter finds an internal problem, but when this error is encountered the Python interpreter does not exit.
SystemExit	Raised when Python interpreter is quit by using the <code>sys.exit()</code> function. If not handled in the code, causes the interpreter to exit.

TypeError	Raised when an operation or function is attempted that is invalid for the specified data type.
ValueError	Raised when the built-in function for a data type has the valid type of arguments, but the arguments have invalid values specified.
RuntimeError	Raised when a generated error does not fall into any category.
NotImplementedError	Raised when an abstract method that needs to be implemented in an inherited class is not actually implemented.

Reading and Writing Files:

The *file* object provides a set of access methods to make our lives easier. We would see how to use *read()* and *write()* methods to read and write files.

The *write()* Method:

- The *write()* method writes any string to an open file. It is important to note that Python strings can have binary data and not just text.
- The *write()* method does not add a newline character ('\n') to the end of the string:

Syntax:

```
fileObject.write(string);
```

Argument of an Exception:

An exception can have an *argument*, which is a value that gives additional information about the problem. The contents of the argument vary by exception. You capture an exception's argument by supplying a variable in the except clause as follows:

```
try:
```

```
    You do your operations here;
```

```
    .....
```

```
except ExceptionType, Argument:
```

```
    You can print value of Argument here...
```

If you are writing the code to handle a single exception, you can have a variable follow the name of the exception in the except statement. If you are trapping multiple exceptions, you can have a variable follow the tuple of the exception.

This variable will receive the value of the exception mostly containing the cause of the exception. The variable can receive a single value or multiple values in the form of a tuple. This tuple usually contains the error string, the error number, and an error location.

Example:

Following is an example for a single exception:

```
def temp_convert(var):  
    try:  
        return int(var)  
    except ValueError, Argument:  
        print "The argument does not contain numbers\n", Argument  
temp_convert("xyz");
```

This would produce following result:

```
The argument does not contain numbers  
invalid literal for int() with base 10: 'xyz'
```

Raising an exceptions:

You can raise exceptions in several ways by using the raise statement. The general syntax for the raise statement.

Syntax:

```
raise [Exception [, args [, traceback]]]
```

Here *Exception* is the type of exception (for example, `NameError`) and *argument* is a value for the exception argument. The argument is optional; if not supplied, the exception argument is `None`.

The final argument, `traceback`, is also optional (and rarely used in practice), and, if present, is the traceback object used for the exception

Example:

```
def functionName( level ):
    if level < 1:
        raise "Invalid level!", level
    # The code below to this would not be executed
    # if we raise the exception
```

Note: In order to catch an exception, an "except" clause must refer to the same exception thrown either class object or simple string. For example to capture above exception we must write our except clause as follows:

```
try:  
    Business Logic here...  
except "Invalid level!":  
    Exception handling here...  
else:  
    Rest of the code here...
```

User-Defined Exceptions:

- Python also allows you to create your own exceptions by deriving classes from the standard built-in exceptions.
- Here is an example related to *RuntimeError*. Here a class is created that is subclassed from *RuntimeError*. This is useful when you need to display more specific information when an exception is caught.
- In the try block, the user-defined exception is raised and caught in the except block. The variable *e* is used to create an instance of the class *Networkerror*.

```
class Networkerror(RuntimeError):  
    def __init__(self, arg):  
        self.args = arg
```

- So once you defined above class, you can raise your exception as follows:

```
try:  
    raise Networkerror("Bad hostname")  
except Networkerror,e:  
    print e.args
```

